

# Examen Primera Convocatoria, Franja Horaria Mañana

```
# Your imports HERE !!!!
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

## 01 - 1.5pt

- Utilizando Matplotlib y Seaborn, crea un gráfico de dispersión (scatter plot) que muestre la relación entre las columnas 'A' y 'B' del siguiente DataFrame. Además, ajusta una línea de regresión a los datos.

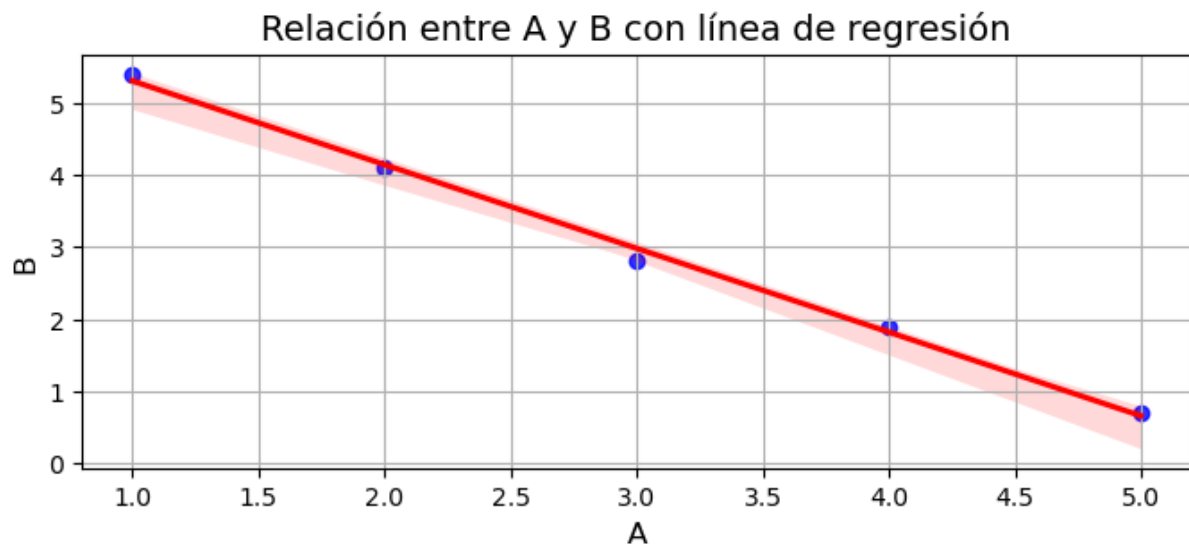
```
# Data
df = pd.DataFrame({'A': [1, 2, 3, 4, 5], 'B': [5.4, 4.1, 2.8, 1.9, 0.7]})

# Your solution HERE !!!!
df = pd.DataFrame(df)

# Crear el gráfico de dispersión y ajustar una línea de regresión
plt.figure(figsize=(8, 3))
sns.regplot(x='A', y='B', data=df, scatter_kws={'color': 'blue'},
            line_kws={'color': 'red'})

# Personalizar el gráfico
plt.title("Relación entre A y B con línea de regresión", fontsize=14)
plt.xlabel("A", fontsize=12)
plt.ylabel("B", fontsize=12)
plt.grid(True)

# Mostrar el gráfico
plt.show()
```



## 02 - 1.5pt

- Escribe una función que reciba un DataFrame, le agregue una nueva columna llamada 'Promedio' que sea el promedio de sus columnas. Luego, filtre las filas donde el valor de 'Promedio' sea mayor que 5 y lo ordene por la columna 'Promedio' en orden descendente.

```
# Your solution HERE !!!!
def func_mean(df):
    # Agregar la columna 'Promedio' calculando el promedio de cada fila
    df['Promedio'] = df.mean(axis=1)

    # Filtrar filas donde el 'Promedio' sea mayor que 5
    filtered_df = df[df['Promedio'] > 5]

    # Ordenar por la columna 'Promedio' en orden descendente
    sorted_df = filtered_df.sort_values(by='Promedio',
ascending=False)

    sorted_df = sorted_df.reset_index(drop=True)

    return sorted_df

# Test Use Case
source_dataframe = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 7, 6], 'C': [7, 12, 9]})
expected_dataframe = pd.DataFrame({'A': [2, 3], 'B': [7, 6], 'C': [12, 9], 'Promedio': [7.0, 6.0]})

output = func_mean(source_dataframe) # This calls your function named func_mean
display(expected_dataframe)
```

```
display(output)
```

```
assert(expected_dataframe.equals(output)) # This will fail if the
dataframe is wrong
```

|   | A | B | C  | Promedio |
|---|---|---|----|----------|
| 0 | 2 | 7 | 12 | 7.0      |
| 1 | 3 | 6 | 9  | 6.0      |

|   | A | B | C  | Promedio |
|---|---|---|----|----------|
| 0 | 2 | 7 | 12 | 7.0      |
| 1 | 3 | 6 | 9  | 6.0      |

### 03 - 1.5pt

- Escribe una función que cada vez que se le invoque devuelva el factorial del siguiente número entero, empezando por 1, hasta cualquier número infinito

```
# Your solution HERE !!!!
```

```
def fact_gen():
    n = 1
    factorial = 1
    while True:
        yield factorial
        n += 1
        factorial *= n
```

```
# Test Use Case
```

```
expected_output = [1, 2, 6, 24, 120, 720, 5040, 40320]
```

```
g = fact_gen() # This calls your function named fact_gen
output = [next(g) for n in range(len(expected_output))]
```

```
display(expected_output)
```

```
display(output)
```

```
assert(expected_output == output) # This will fail if the list is
wrong
```

```
[1, 2, 6, 24, 120, 720, 5040, 40320]
```

```
[1, 2, 6, 24, 120, 720, 5040, 40320]
```

### 04 - 1.5pt

- Crear una función que reciba un array de NumPy y que reemplace todos los números primos por sus raíces cuadradas
- Primo es aquel número natural mayor que 1, que solo es divisible por 1 y por si mismo

```
# Your solution HERE !!!!
```

```
def is_prime(n):
```

```

if n <= 1:
    return False
for i in range(2, int(np.sqrt(n)) + 1):
    if n % i == 0:
        return False
return True

def prime_per_sqrt(array):
    # Convertir el array a DataFrame
    df = pd.DataFrame(array)
    # Aplicar la función `is_prime` para identificar primos en cada
    elemento
    is_prime_mask = df.map(is_prime)
    # Reemplazar números primos con su raíz cuadrada
    df = df.where(~is_prime_mask, np.sqrt(df).round(2))

    # Convertir de vuelta a un array de NumPy
    return df.to_numpy()

# Test Use Case
source_array = np.array([[20, 17, 33, 11], [3, 25, 5, 49], [18, 28,
32, 2], [9, 1, 23, 40]])
expected_output = np.array([[20, 4.12, 33, 3.31], [1.73, 25, 2.23,
49], [18, 28, 32, 1.41], [9, 1, 4.79, 40]])

output = prime_per_sqrt(source_array) # This calls your function named
prime_per_sqrt

display(expected_output)
display(output)

assert(np.allclose(output, expected_output, atol = 0.01)) # This will
fail if the array is wrong

array([[20.  ,  4.12, 33.  ,  3.31],
       [ 1.73, 25.  ,  2.23, 49.  ],
       [18.  , 28.  , 32.  ,  1.41],
       [ 9.  ,  1.  ,  4.79, 40.  ]])

array([[20.  ,  4.12, 33.  ,  3.32],
       [ 1.73, 25.  ,  2.24, 49.  ],
       [18.  , 28.  , 32.  ,  1.41],
       [ 9.  ,  1.  ,  4.8 , 40.  ]])

```

## 05 - 4pt

El conjunto de datos "estaciones\_bici.csv" (fichero disponible adjunto al notebook) proviene de una descarga de datos del servicio web de la empresa municipal Valenbici, dedicada al alquiler de bicicletas en Valencia.

Los datos recabados de este servicio web son de mediciones cada 10 minutos de las estaciones:

<https://valencia.opendatasoft.com/explore/dataset/valenbisi-disponibilitat-valenbisi-disponibilidad/information/>

Cada estación está compuesta por un número variable de bornetas (total) donde se pueden anclar las bicicletas. Los datos obtenidos de cada estación (registros) refleja el número de bornetas libres (free) y el número de bicicletas disponibles (available).

### Columnas para trabajar:

- station: id de la estación
- total: número total de bornetas
- updated: timestamp del estado en de bicis en la estación
- available: número de bicis disponibles

### Preguntas

1. Carga de datos de csv (estaciones\_bici.csv) en un DataFrame, descripción de los datos
2. ¿Cuál es el rango temporal del dataset? Obtención del número de estaciones que tienen un total de 25 bornetas, y cuales son
3. Número de estación con la media más baja de bicis disponibles
4. Realizar el histograma de bicis disponibles de la estación del punto anterior
5. Realizar gráfica con la línea temporal de bicis disponibles de la estación del punto anterior. Pista, es necesario cambiar el índice a uno tipo fecha

```
# Your solution HERE !!!!
```

```
#1. Carga de datos de csv (estaciones_bici.csv) en un DataFrame, descripción de los datos
```

```
# Cargar el archivo CSV
```

```
data = pd.read_csv('excel.csv', delimiter=';')  
display(data)
```

```
# Convertir la columna 'updated' a tipo datetime
```

```
data['updated'] = pd.to_datetime(data['updated'])
```

```
# Descripción de los datos
```

```
description = data.describe(include='all')
```

```
display("Descripción de los datos:", description)
```

|       | Unnamed: 0 | _id                      | available | connected | \ |
|-------|------------|--------------------------|-----------|-----------|---|
| 0     | 0          | 5c6050a42554172704fccdc0 | 9         | 1         |   |
| 1     | 1          | 5c6050a42554172704fccdc1 | 6         | 1         |   |
| 2     | 2          | 5c605be225541729b7d50885 | 20        | 1         |   |
| 3     | 3          | 5c605be225541729b7d50886 | 6         | 1         |   |
| 4     | 4          | 5c605be225541729b7d50887 | 9         | 1         |   |
| ...   | ...        | ...                      | ...       | ...       |   |
| 27542 | 27542      | 5c61face25541729b7d57419 | 0         | 1         |   |
| 27543 | 27543      | 5c61face25541729b7d5741a | 15        | 1         |   |
| 27544 | 27544      | 5c61face25541729b7d5741b | 1         | 1         |   |

|       |       |                          |   |   |
|-------|-------|--------------------------|---|---|
| 27545 | 27545 | 5c61face25541729b7d5741c | 1 | 1 |
| 27546 | 27546 | 5c61face25541729b7d5741d | 8 | 1 |

|       |            | download_date | station | free | open | ticket | total | \ |
|-------|------------|---------------|---------|------|------|--------|-------|---|
| 0     | 2019-02-10 | 17:25:37.787  | 64      | 11   | 1    | 0      | 20    |   |
| 1     | 2019-02-10 | 17:25:37.787  | 73      | 14   | 1    | 1      | 20    |   |
| 2     | 2019-02-10 | 18:13:39.827  | 63      | 0    | 1    | 1      | 20    |   |
| 3     | 2019-02-10 | 18:13:39.827  | 64      | 14   | 1    | 0      | 20    |   |
| 4     | 2019-02-10 | 18:13:39.827  | 65      | 10   | 1    | 1      | 19    |   |
| ...   |            |               |         |      |      |        |       |   |
| 27542 | 2019-02-11 | 23:44:00.786  | 260     | 20   | 1    | 0      | 20    |   |
| 27543 | 2019-02-11 | 23:44:00.786  | 261     | 4    | 1    | 0      | 19    |   |
| 27544 | 2019-02-11 | 23:44:00.786  | 268     | 9    | 1    | 1      | 10    |   |
| 27545 | 2019-02-11 | 23:44:00.786  | 269     | 14   | 1    | 0      | 15    |   |
| 27546 | 2019-02-11 | 23:44:00.786  | 276     | 12   | 1    | 1      | 20    |   |

|       |            | updated      |
|-------|------------|--------------|
| 0     | 2019-02-10 | 17:21:13.000 |
| 1     | 2019-02-10 | 17:24:13.000 |
| 2     | 2019-02-10 | 18:09:16.000 |
| 3     | 2019-02-10 | 18:12:15.000 |
| 4     | 2019-02-10 | 18:09:16.000 |
| ...   |            |              |
| 27542 | 2019-02-11 | 23:42:16.000 |
| 27543 | 2019-02-11 | 23:39:16.000 |
| 27544 | 2019-02-11 | 23:42:16.000 |
| 27545 | 2019-02-11 | 23:39:16.000 |
| 27546 | 2019-02-11 | 23:42:16.000 |

[27547 rows x 11 columns]

'Descripción de los datos:'

|             | Unnamed: 0   | _id                      | available    |
|-------------|--------------|--------------------------|--------------|
| connected \ |              |                          |              |
| count       | 27547.000000 | 27547                    | 27547.000000 |
| unique      | NaN          | 27547                    | NaN          |
| NaN         |              |                          |              |
| top         | NaN          | 5c6050a42554172704fccdc0 | NaN          |
| NaN         |              |                          |              |
| freq        | NaN          | 1                        | NaN          |
| NaN         |              |                          |              |
| mean        | 13773.000000 | NaN                      | 8.974444     |
| 0.996261    |              |                          |              |
| min         | 0.000000     | NaN                      | 0.000000     |
| 0.000000    |              |                          |              |
| 25%         | 6886.500000  | NaN                      | 3.000000     |
| 1.000000    |              |                          |              |
| 50%         | 13773.000000 | NaN                      | 8.000000     |

```

1.000000
75%      20659.500000      NaN      14.000000
1.000000
max      27546.000000      NaN      40.000000
1.000000
std      7952.278269      NaN      7.307137
0.061035

```

```

              download_date      station      free
open \
count      27547  27547.000000  27547.000000  27547.0
unique      105      NaN      NaN      NaN
top  2019-02-11 09:14:47.430      NaN      NaN      NaN
freq      276      NaN      NaN      NaN
mean      NaN  138.449196  10.629397  1.0
min      NaN  1.000000  0.000000  1.0
25%      NaN  69.000000  4.000000  1.0
50%      NaN  139.000000  10.000000  1.0
75%      NaN  207.000000  15.000000  1.0
max      NaN  276.000000  40.000000  1.0
std      NaN  79.657747  7.492671  0.0

```

```

              ticket      total      updated
count  27547.000000  27547.000000      27547
unique      NaN      NaN      NaN
top      NaN      NaN      NaN
freq      NaN      NaN      NaN
mean    0.503794  19.915381  2019-02-11 08:57:02.201437184
min    0.000000  10.000000      2019-02-10 17:21:13
25%    0.000000  15.000000      2019-02-11 01:27:15
50%    1.000000  20.000000      2019-02-11 08:57:14
75%    1.000000  20.000000      2019-02-11 16:27:15
max    1.000000  40.000000      2019-02-11 23:42:16
std    0.499995  5.570912      NaN

```

```

# 2. ¿Cuál es el rango temporal del dataset? Obtención del número de
estaciones que tienen un total de 25 bornetas, y cuales son
# Rango temporal del dataset
time_range = data['updated'].min(), data['updated'].max()

```

```
# Número de estaciones con un total de 25 bornetas
stations_25_bornetas = data[data['total'] == 25]['station'].unique()
num_stations_25_bornetas = len(stations_25_bornetas)
```

```
print("Rango temporal del dataset:", time_range)
print(f"Número de estaciones con 25 bornetas:
{num_stations_25_bornetas}")
print ("Estaciones con 25 bornetas:", stations_25_bornetas)
```

```
Rango temporal del dataset: (Timestamp('2019-02-10 17:21:13'),
Timestamp('2019-02-11 23:42:16'))
Número de estaciones con 25 bornetas: 20
Estaciones con 25 bornetas: [ 66  69  75  80 150  49  52  53  55  83
106 124 128 142   9  28  36   1
   4 217]
```

```
#3. Número de estación con la media más baja de bicis disponibles
station_mean_bikes = data.groupby('station')['available'].mean()
station_lowest_mean = station_mean_bikes.idxmin()
```

```
print("Estación con la media más baja de bicis disponibles:",
station_lowest_mean)
```

```
Estación con la media más baja de bicis disponibles: 268
```

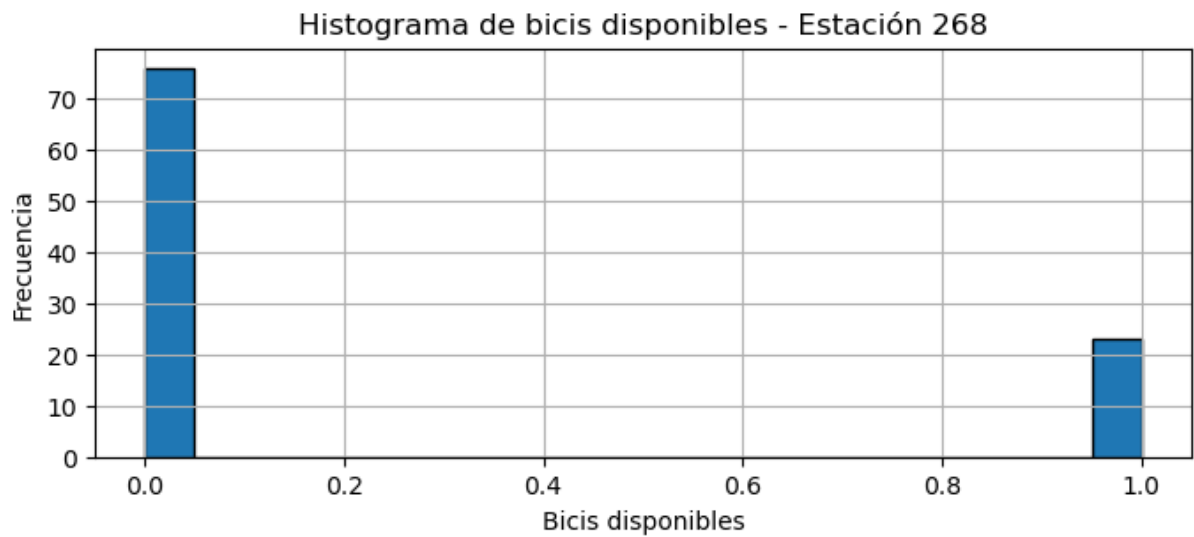
```
#4. Realizar el histograma de bicis disponibles de la estación del
punto anterior
```

```
# Filtrar datos para la estación con la media más baja
data_station_lowest_mean = data[data['station'] ==
station_lowest_mean]
```

```
# Histograma de bicis disponibles
```

```
plt.figure(figsize=(8, 3))
plt.hist(data_station_lowest_mean['available'], bins=20,
edgecolor='k')
plt.title(f"Histograma de bicis disponibles - Estación
{station_lowest_mean}")
plt.xlabel("Bicis disponibles")
plt.ylabel("Frecuencia")
plt.grid(True)
plt.show()
```





#5. Realizar gráfica con la línea temporal de bicis disponibles de la estación del punto anterior. Pista, es necesario cambiar el índice a uno tipo fecha

```
# Línea temporal de bicis disponibles
data_station_lowest_mean.set_index('updated', inplace=True)
plt.figure(figsize=(12, 3))
plt.plot(data_station_lowest_mean['available'], label=f'Estación
{station_lowest_mean}', linewidth=1.5)
plt.title(f"Línea temporal de bicis disponibles - Estación
{station_lowest_mean}")
plt.xlabel("Tiempo")
plt.ylabel("Bicis disponibles")
plt.legend()
plt.grid(True)
plt.show()
```

